Imports

In [ ]:
```
# Standard libraries
import numpy as np
import pandas as pd

# Visualization
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

# Statistical modeling
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Scikit-learn - Preprocessing
from sklearn.preprocessing import StandardScaler

# Scikit-learn - Model selection and splitting
from sklearn.model_selection import train_test_split

# Scikit-learn - Feature selection
from sklearn.feature_selection import RFE

# Scikit-learn - Models
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

# Scikit-learn - Metrics
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

Data loading & preprocessing

In [126…
```
df = pd.read_csv("insurance.csv")
```

In [127…
```
df.head().T
```

Out[127…

	0	1	2	3	4
age	19	18	28	33	32
sex	female	male	male	male	male
bmi	27.9	33.8	33.0	22.7	28.9
children	0	1	3	0	0
smoker	yes	no	no	no	no
region	southwest	southeast	southeast	northwest	northwest
expenses	16884.92	1725.55	4449.46	21984.47	3866.86

In [128…
```
df.shape
```

Out[128…
```
(1338, 7)
```

In [129…
```
df.isna().sum() # No missing values
```

Out[129…
```
age         0
sex         0
bmi         0
children    0
smoker      0
region      0
expenses    0
dtype: int64
```

In [130…
```
# Find duplicate rows (i.e., all columns are the same), keep=False marks all dup
duplicate_rows = df[df.duplicated(keep=False)]

# Show duplicate rows (if any)
if not duplicate_rows.empty:
    print("Duplicate rows (showing all instances of each duplicate):")
    display(duplicate_rows)
    print(f"Number of duplicate rows (all instances): {len(duplicate_rows)}")
else:
    print("No duplicate rows found.")
```

Duplicate rows (showing all instances of each duplicate):

	age	sex	bmi	children	smoker	region	expenses
195	19	male	30.6	0	no	northwest	1639.56
581	19	male	30.6	0	no	northwest	1639.56

Number of duplicate rows (all instances): 2

In [131…
```
# Remove duplicated rows from the dataframe and keep the first occurrence
df = df.drop_duplicates(keep="first").reset_index(drop=True)
```

In [132…
```
df.shape
```

Out[132…
```
(1337, 7)
```

In [133…
```
all_columns         = list(df)
numeric_columns     = ['age', 'bmi', 'children', 'expenses']
categorical_columns = [x for x in all_columns if x not in numeric_columns ]

print('\nNumeric columns')
print(numeric_columns)
print('\nCategorical columns')
print(categorical_columns)
```

```
Numeric columns
['age', 'bmi', 'children', 'expenses']

Categorical columns
['sex', 'smoker', 'region']
```

In [134…
```
df.describe().T
```

Out[134…

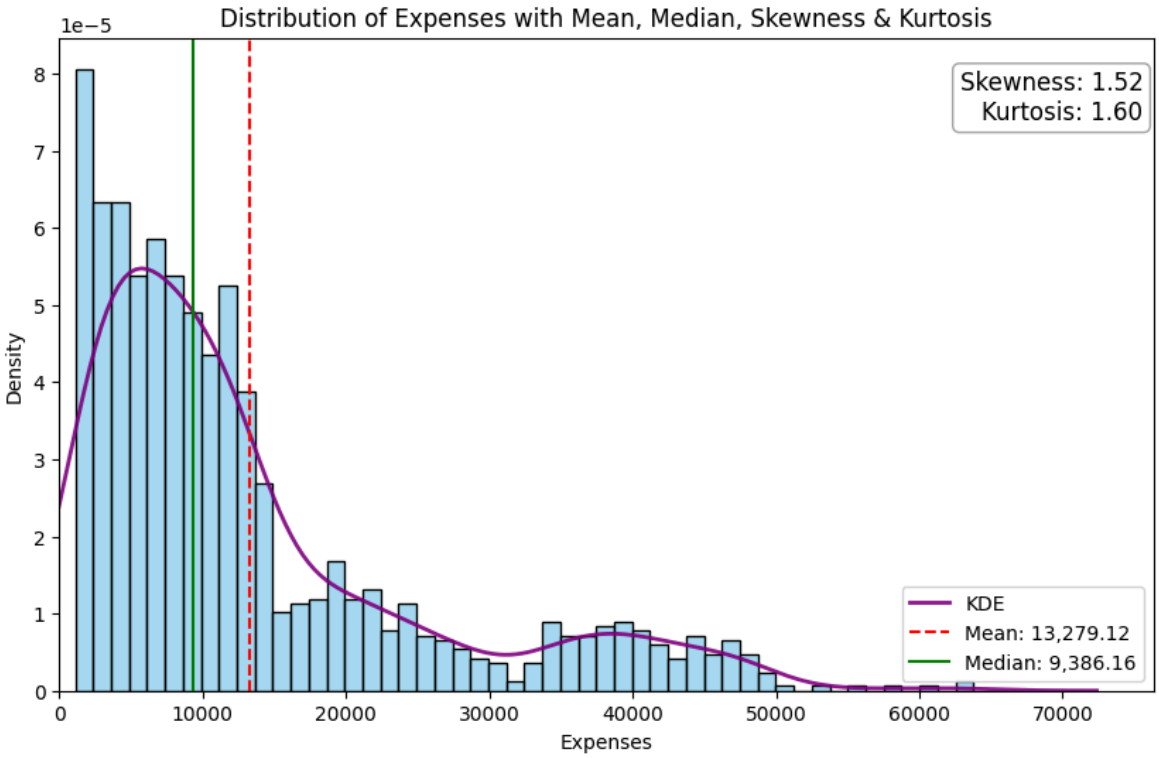
	count	mean	std	min	25%	50%	75%	
age	1337.0	39.222139	14.044333	18.00	27.00	39.00	51.00	6
bmi	1337.0	30.665520	6.100664	16.00	26.30	30.40	34.70	5
children	1337.0	1.095737	1.205571	0.00	0.00	1.00	2.00	
expenses	1337.0	13279.121638	12110.359657	1121.87	4746.34	9386.16	16657.72	6377

Data visualisation

Expenses visualisation

In [135…

```
plt.figure(figsize=(10,6))
sns.histplot(df['expenses'], bins=50, kde=False, color='skyblue', edgecolor='bla

sns.kdeplot(df['expenses'], color='purple', linewidth=2, label='KDE', alpha=0.9)

mean_expenses = df['expenses'].mean()
median_expenses = df['expenses'].median()

plt.axvline(mean_expenses, color='red', linestyle='--', label=f'Mean: {mean_expe
plt.axvline(median_expenses, color='green', linestyle='-', label=f'Median: {medi

skewness = df['expenses'].skew()
kurtosis = df['expenses'].kurtosis()

plt.text(0.99, 0.95,
         f'Skewness: {skewness:.2f}\nKurtosis: {kurtosis:.2f}',
         transform=plt.gca().transAxes,
         fontsize=12, va='top', ha='right',
         bbox=dict(boxstyle="round,pad=0.3", edgecolor="gray", facecolor="white"

plt.xlim(0, None)
plt.legend()
plt.xlabel('Expenses')
plt.ylabel('Density')
plt.title('Distribution of Expenses with Mean, Median, Skewness & Kurtosis')
plt.show()
```



The distribution of insurance expenses is **highly skewed to the right**, as shown by the positive skewness value and can be seen in both the histogram and KDE plot. This means that while most people have **relatively low insurance expenses**, a smaller number have **very high expenses**.

The mean insurance expense is much higher than the median, highlighting the impact of outliers—those extreme high expenses pull up the average.

Finally, the **high kurtosis** tells us that the distribution has **heavier tails** than a normal distribution. In other words, there are more outliers (very high expense values) than would be expected if expenses followed a normal bell curve.

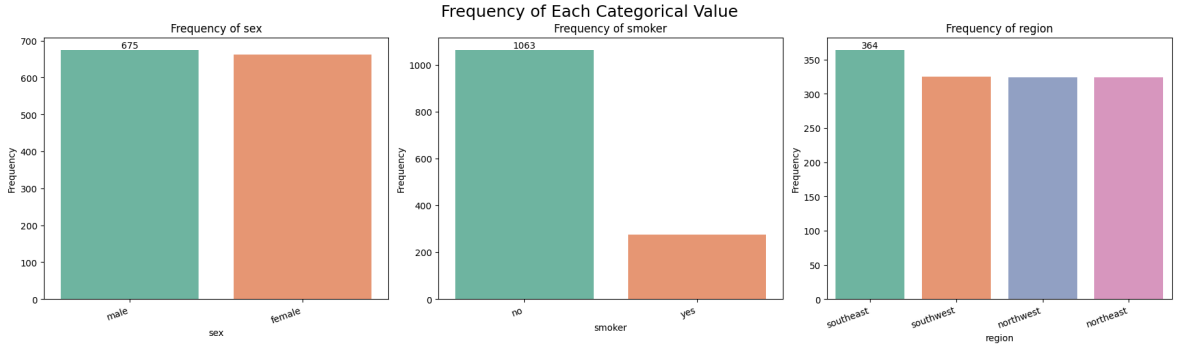
Visualising categorical data

In [136…
```
# Display unique values for each categorical column
print("Categorical column values:")
for col in categorical_columns:
    unique_vals = df[col].unique()
    print(f"- {col}: {unique_vals}")
```

```
Categorical column values:
- sex: ['female' 'male']
- smoker: ['yes' 'no']
- region: ['southwest' 'southeast' 'northwest' 'northeast']
```

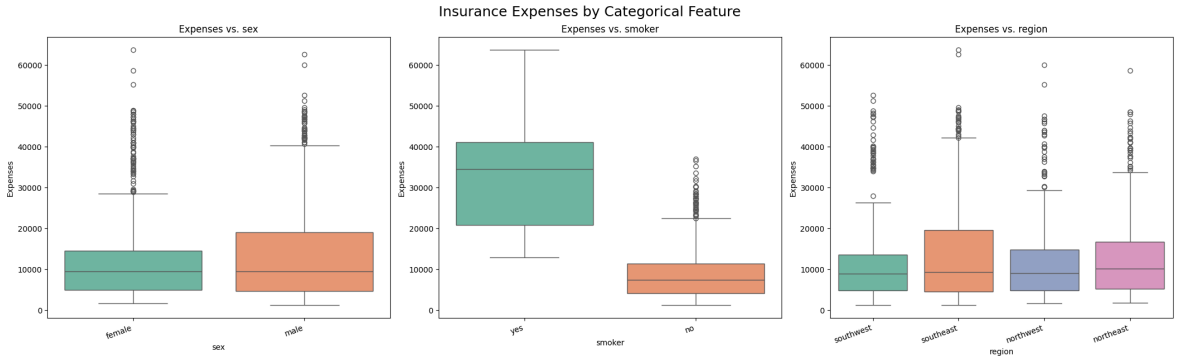
In [137…
```
num_cats = len(categorical_columns)
fig, axes = plt.subplots(1, num_cats, figsize=(6*num_cats, 5), constrained_layou

if num_cats == 1:
    axes = [axes]

for idx, col in enumerate(categorical_columns):
    ax = axes[idx]
```

```
    value_counts = df[col].value_counts()
    sns.barplot(x=value_counts.index, y=value_counts.values, ax=ax, hue=value_co
    ax.set_title(f"Frequency of {col}")
    ax.set_xlabel(col)
    ax.set_ylabel("Frequency")
    ax.bar_label(ax.containers[0])
    plt.setp(ax.get_xticklabels(), rotation=20, ha='right')

plt.suptitle('Frequency of Each Categorical Value', fontsize=18, y=1.05)
plt.show()
```



In [138…

```
# Box plots of expenses (price) vs. each categorical value
fig, axes = plt.subplots(1, len(categorical_columns), figsize=(7*len(categorical

if len(categorical_columns) == 1:
    axes = [axes]

for idx, col in enumerate(categorical_columns):
    ax = axes[idx]
    # Pass x variable also to 'hue' and set legend=False, per warning recommenda
    sns.boxplot(x=col, y='expenses', data=df, ax=ax, hue=col, palette="Set2", le
    ax.set_title(f"Expenses vs. {col}")
    ax.set_xlabel(col)
    ax.set_ylabel("Expenses")
    plt.setp(ax.get_xticklabels(), rotation=20, ha='right')

plt.suptitle('Insurance Expenses by Categorical Feature', fontsize=18, y=1.05)
plt.show()
```



Interpretation of Boxplots Above

The boxplots above display the distribution of insurance expenses for each categorical feature in the dataset. Here are some observations and interpretations:

- **Sex**: The median expenses for males and females are quite similar, suggesting that gender alone does not have a strong direct effect on insurance charges. However,

the spread of expenses, especially among males, appears slightly wider, reflecting some outliers.

- **Smoker**: There is a dramatic difference in expenses between smokers and non-smokers. Smokers have significantly higher median expenses and a much wider range, indicating that smoking status is a major driver of insurance costs in this dataset.

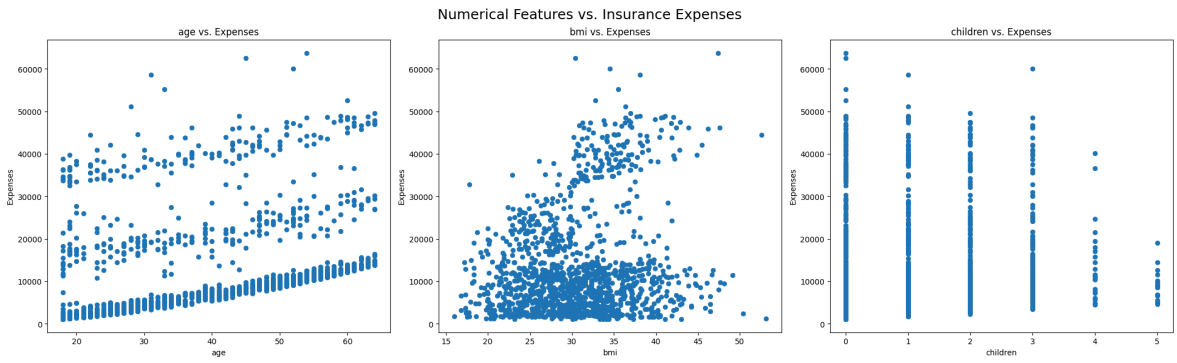
- **Region**: While there are some variations in the medians and spreads, the differences in insurance expenses by region are less pronounced than for smoking status. This suggests region has less impact compared to other factors, though certain regions may have more high-expense outliers.

Overall, the boxplots highlight that **smoker status** is the most significant categorical variable influencing insurance expenses, with smokers incurring much higher costs. Other categorical variables, like sex and region, appear to have smaller effects.

Visualising numerical data

In [139…

```
# Scatter plots of each numerical feature vs. expenses
numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns.tolis
numerical_columns = [col for col in numerical_columns if col != 'expenses']

num_numeric = len(numerical_columns)
fig, axes = plt.subplots(1, num_numeric, figsize=(7*num_numeric, 6), constrained

if num_numeric == 1:
    axes = [axes]

for idx, col in enumerate(numerical_columns):
    ax = axes[idx]
    sns.scatterplot(x=df[col], y=df['expenses'], ax=ax, color='C0', edgecolor=No
    ax.set_xlabel(col)
    ax.set_ylabel('Expenses')
    ax.set_title(f"{col} vs. Expenses")

plt.suptitle('Numerical Features vs. Insurance Expenses', fontsize=18, y=1.05)
plt.show()
```



Interpretation of Scatterplots Above

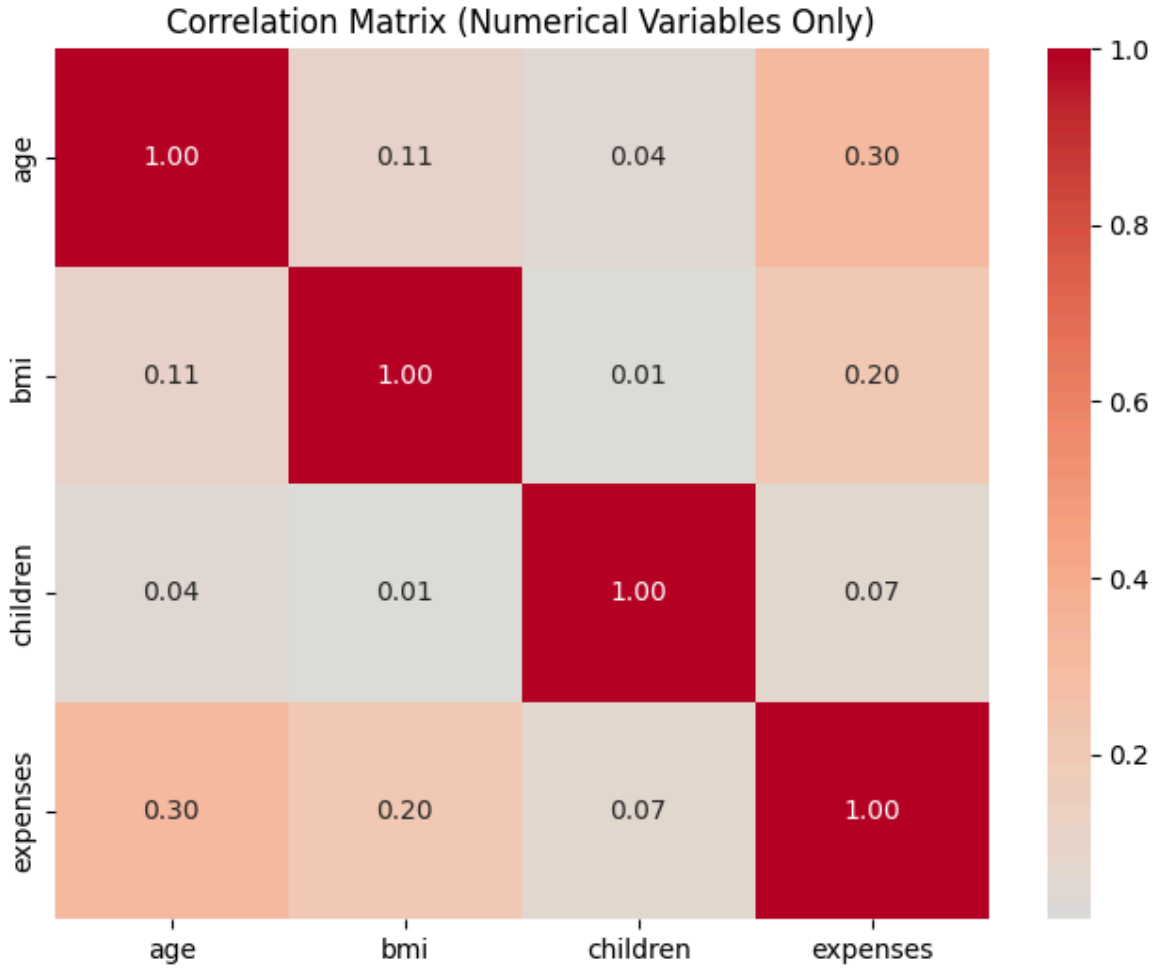
The scatterplots shown above illustrate the relationships between each numerical feature and insurance expenses. Notable patterns include:

- **Age**: There is a positive trend between age and insurance expenses. Generally, as age increases, expenses tend to rise, with more noticeable jumps in expenses among older individuals, possibly due to increased health risks.
- **BMI**: Higher BMI shows some association with higher insurance expenses, though the relationship is less clear-cut compared to age. There are several individuals with high BMI and high expenses, possibly reflecting higher health risks for those with higher BMI.
- **Children**: The number of children covered by insurance does not show a strong or consistent relationship with expenses. While expenses appear scattered across different numbers of children, there's no clear trend indicating that having more children leads to higher expenses.

Overall, **age** has the strongest positive correlation with insurance expenses among the numerical features, followed by BMI, while the number of children appears to have minimal impact.

In [140…

```
num_corr = df[numeric_columns].corr()

plt.figure(figsize=(8,6))
sns.heatmap(num_corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Correlation Matrix (Numerical Variables Only)")
plt.show()
```



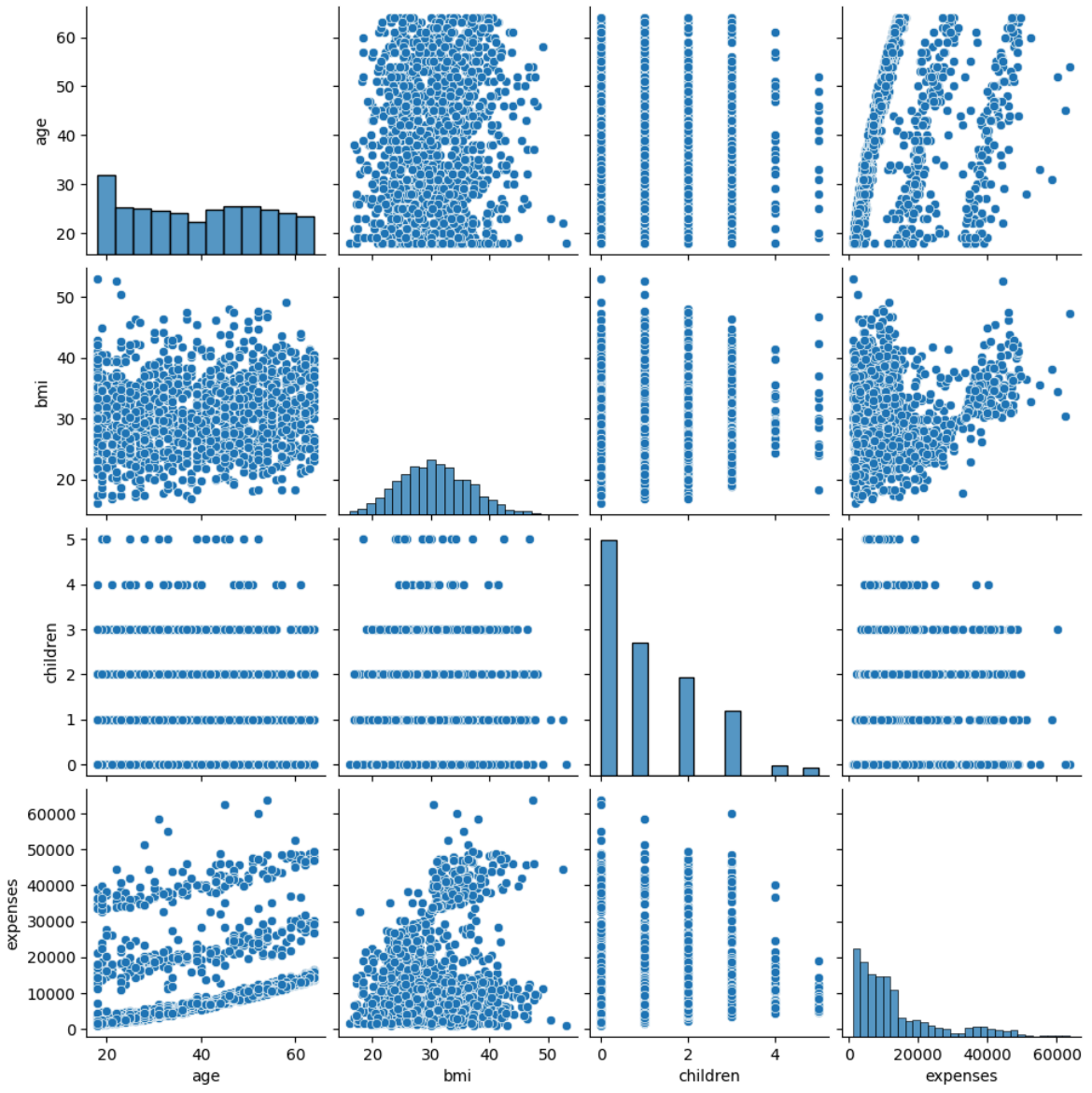
Interpretation of Correlation Matrix Above

- **Age** and **expenses** show a moderately strong positive correlation, supporting the trend seen in the scatterplots—older individuals tend to have higher insurance costs.
- **BMI** also exhibits a positive correlation with expenses, but not as pronounced as age. This indicates that higher BMI is generally associated with higher medical expenses, though other factors may also play a role.
- **Children** has a very weak (near zero) correlation with expenses, echoing our earlier observation that the number of children does not substantially influence insurance costs.
- Correlations among features like age, bmi, and children themselves are all low, suggesting these variables are largely independent in this dataset.

In summary, age is the most influential numerical variable related to expenses, with BMI being a secondary factor. Number of children plays a minor role.

In [141…

```
sns.pairplot(df)
plt.show()
```



In [142...
```
def dummies(x, df):
    temp = pd.get_dummies(df[x], prefix=x, drop_first=True).astype(int)
    df = pd.concat([df, temp], axis=1)
    df.drop([x], axis=1, inplace=True)
    return df

df = dummies('sex', df)
df = dummies('smoker', df)
df = dummies('region', df)
df.head()
```

Out[142...

	age	bmi	children	expenses	sex_male	smoker_yes	region_northwest	region_south
0	19	27.9	0	16884.92	0	1	0	
1	18	33.8	1	1725.55	1	0	0	
2	28	33.0	3	4449.46	1	0	0	
3	33	22.7	0	21984.47	1	0	1	
4	32	28.9	0	3866.86	1	0	1	

Selecting the features

In [143...
```
np.random.seed(42)
df_train, df_test = train_test_split(df, train_size = 0.75, test_size = 0.25, ra

#Dividing data into X and y variables
y_train = df_train.pop('expenses')
X_train = df_train

y_test = df_test.pop('expenses')
X_test = df_test
```

In [144...
```
df_train.head()
```

Out[144...

	age	bmi	children	sex_male	smoker_yes	region_northwest	region_southeast	r
762	27	26.0	0	1	0	0	0	
1078	63	33.7	3	1	0	0	1	
178	46	28.9	2	0	0	0	0	
287	63	26.2	0	0	0	1	0	
1289	38	20.0	2	0	0	0	0	

In [145...
```
numeric_columns_without_expenses = ['age', 'bmi', 'children']

scaler_std = StandardScaler()
X_train[numeric_columns_without_expenses] = scaler_std.fit_transform(X_train[num
```

In [146...
```
X_train.head()
```

Out[146...

	age	bmi	children	sex_male	smoker_yes	region_northwest	region_
762	-0.862948	-0.767716	-0.912228	1	0	0	
1078	1.715175	0.513150	1.596607	1	0	0	
178	0.497728	-0.285312	0.760329	0	0	0	
287	1.715175	-0.734447	-0.912228	0	0	1	
1289	-0.075188	-1.765793	0.760329	0	0	0	

In [147...

```
rfe = RFE(estimator=LinearRegression(), n_features_to_select=4)
rfe = rfe.fit(X_train, y_train)
```

In [148...

```
# Display selected features along with their rankings
feature_ranking = list(zip(X_train.columns, rfe.ranking_))
selected_features = [feature for feature, rank in feature_ranking if rank == 1]
print("Selected features (ranking = 1):", selected_features)

print("\nAll feature rankings:")
for feature, rank in feature_ranking:
    selected_str = "(SELECTED)" if rank == 1 else ""
    print(f"{feature}: ranking = {rank} {selected_str}")
```

```
Selected features (ranking = 1): ['age', 'bmi', 'children', 'smoker_yes']

All feature rankings:
age: ranking = 1 (SELECTED)
bmi: ranking = 1 (SELECTED)
children: ranking = 1 (SELECTED)
sex_male: ranking = 5
smoker_yes: ranking = 1 (SELECTED)
region_northwest: ranking = 4
region_southeast: ranking = 2
region_southwest: ranking = 3
```

In [149...

```
X_train.columns[rfe.support_] # See which features were selected
```

Out[149...

```
Index(['age', 'bmi', 'children', 'smoker_yes'], dtype='object')
```

In [150...

```
X_train_rfe = X_train[X_train.columns[rfe.support_]]
X_train_rfe.head()
```

Out[150...

	age	bmi	children	smoker_yes
762	-0.862948	-0.767716	-0.912228	0
1078	1.715175	0.513150	1.596607	0
178	0.497728	-0.285312	0.760329	0
287	1.715175	-0.734447	-0.912228	0
1289	-0.075188	-1.765793	0.760329	0

Training the model

In [151…
```
def build_model(X,y):
    X = sm.add_constant(X) #Adding the constant for intercept
    lm = sm.OLS(y,X).fit() # fitting the model
    print(lm.summary()) # model summary
    return X

def checkVIF(X):
    vif = pd.DataFrame()
    vif['Features'] = X.columns
    vif['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[
    vif['VIF'] = round(vif['VIF'], 2)
    vif = vif.sort_values(by = "VIF", ascending = False)
    return(vif)
```

In [152…
```
X_train_new = build_model(X_train_rfe, y_train)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:               expenses   R-squared:                       0.729
Model:                            OLS   Adj. R-squared:                  0.728
Method:                 Least Squares   F-statistic:                     670.9
Date:                Mon, 24 Nov 2025   Prob (F-statistic):          6.34e-281
Time:                        09:07:15   Log-Likelihood:                -10156.
No. Observations:                1002   AIC:                         2.032e+04
Df Residuals:                     997   BIC:                         2.035e+04
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       8361.5817    216.863     38.557      0.000    7936.021    8787.142
age         3439.9063    195.135     17.628      0.000    3056.984    3822.829
bmi         1852.8942    194.652      9.519      0.000    1470.920    2234.868
children     635.7646    193.790      3.281      0.001     255.481    1016.049
smoker_yes  2.307e+04    480.113     48.059      0.000    2.21e+04     2.4e+04
==============================================================================
Omnibus:                      248.114   Durbin-Watson:                   1.968
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              590.905
Skew:                           1.322   Prob(JB):                    4.86e-129
Kurtosis:                       5.677   Cond. No.                         2.71
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

In [153…
```
checkVIF(X_train_new)
```

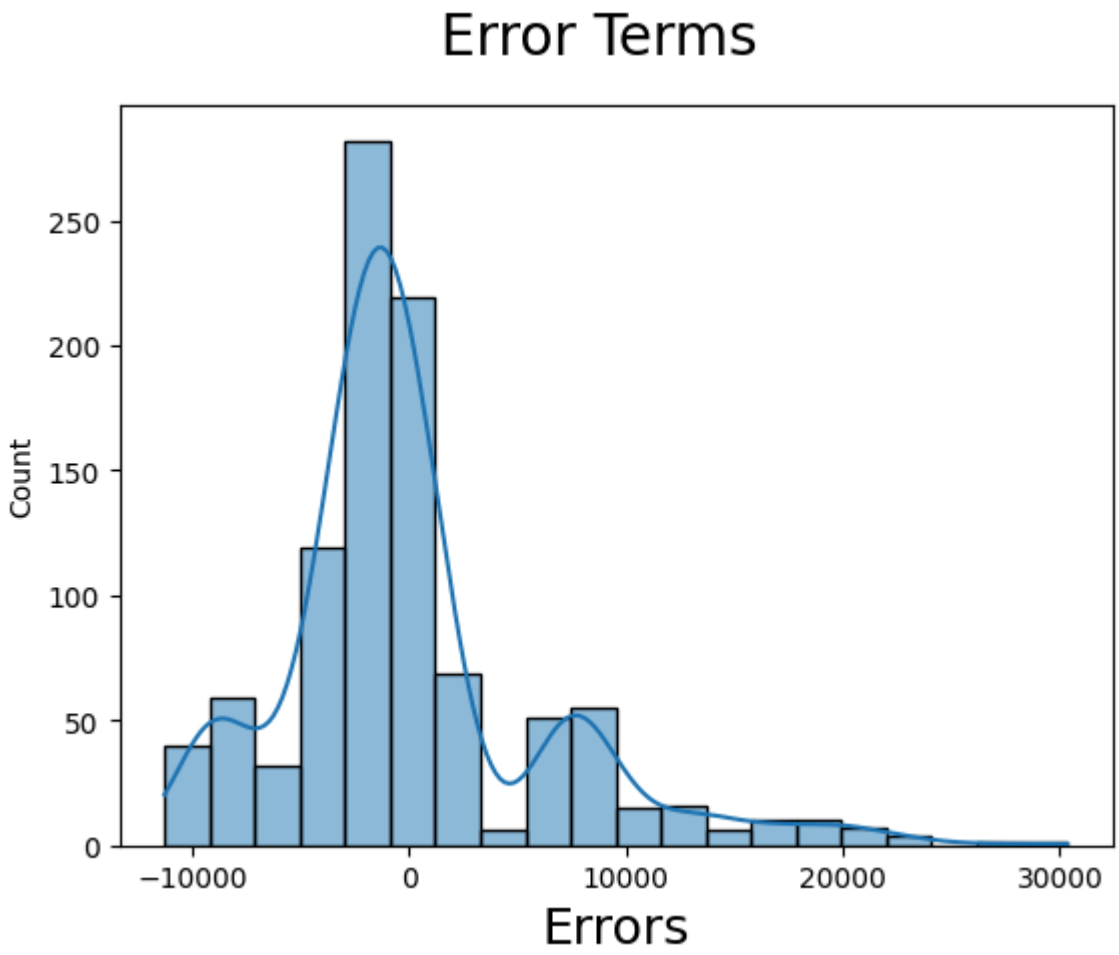
Out[153…

	Features	VIF
0	const	1.26
1	age	1.02
2	bmi	1.01
3	children	1.00
4	smoker_yes	1.00

In [154…
```
lm = sm.OLS(y_train,X_train_new).fit()
y_train_price = lm.predict(X_train_new)
```

In [155…
```
# Plot the histogram of the error terms
fig = plt.figure()
sns.histplot((y_train - y_train_price), bins = 20, kde=True)
fig.suptitle('Error Terms', fontsize = 20)
plt.xlabel('Errors', fontsize = 18)
```

Out[155… `Text(0.5, 0, 'Errors')`



The error terms are **centered around zero**, which is expected and indicates that the model is not systematically over- or under-predicting. However, the distribution of residuals shows **significant deviations from normality**, which violates one of the key assumptions of OLS regression.

Looking at the model statistics:

- **Skewness: 1.322** - The residuals are **positively skewed (right-skewed)**, meaning there are more large positive errors than large negative errors. This can be seen in the histogram as a longer tail on the right side.
- **Kurtosis: 5.677** - The distribution has **heavy tails** (much higher than the normal distribution's kurtosis of 3), indicating the presence of outliers and extreme residual values.
- **Jarque-Bera test p-value: 4.86e-129** - This test **strongly rejects the null hypothesis of normality**, confirming that the residuals are not normally distributed.

- **Omnibus test p-value: 0.000** - Also strongly rejects normality.

Why this matters: The normality assumption is important for OLS regression because it ensures that:

1. Confidence intervals and hypothesis tests are valid
2. The model's predictions are reliable
3. The least squares estimates are the best linear unbiased estimators (BLUE)

Observations:

- The residuals show **significant deviations from normality**, which is **not what we expect** for a well-specified OLS regression model.
- The right-skewed residuals likely reflect the **right-skewed distribution of the target variable (expenses)** that we observed earlier in the data exploration.
- The persistent non-normality may be due to:
  - **Non-linear relationships** between features and the target variable that a linear model cannot capture
  - **Heteroscedasticity** (varying variance of residuals across different values of predictors)
  - **Missing important features** or interactions that would better explain the variance

Potential solutions:

- Consider **transforming the dependent variable** (e.g., log transformation) to address the skewness
- Explore **non-linear models** such as polynomial regression, decision trees, or ensemble methods
- Consider **robust regression techniques** (e.g., Huber regression, RANSAC) that are less sensitive to outliers
- Investigate **heteroscedasticity** and apply appropriate transformations or weighted least squares
- For practical purposes, if the model's R^2 and predictions are acceptable, the violation may be acceptable, though confidence intervals should be interpreted with caution

Predicting on TEST data

In [156…
```
X_test[numeric_columns_without_expenses] = scaler_std.transform(X_test[numeric_c
```

In [157…
```
X_test.head()
```

Out[157…

	age	bmi	children	sex_male	smoker_yes	region_northwest	region_
899	0.712572	-1.349927	-0.912228	1	0	0	
1063	-0.719719	-0.834254	2.432886	0	0	0	
1255	0.855801	0.962284	1.596607	0	0	1	
298	-0.576490	0.629592	1.596607	1	1	1	
237	-0.576490	1.294977	0.760329	1	0	0	

In [158…

```
y_test.head()
```

Out[158…

```
899      8688.86
1063     5708.87
1255    11436.74
298     38746.36
237      4463.21
Name: expenses, dtype: float64
```

In [159…

```
# Get feature names from the FINAL training set
selected_feature_names = X_train_new.drop('const', axis=1).columns

# Select same features from test set
X_test_new = X_test[selected_feature_names]

# Add constant
X_test_new = sm.add_constant(X_test_new)
```

In [160…

```
X_test_new.head()
```

Out[160…

	const	age	bmi	children	smoker_yes
899	1.0	0.712572	-1.349927	-0.912228	0
1063	1.0	-0.719719	-0.834254	2.432886	0
1255	1.0	0.855801	0.962284	1.596607	0
298	1.0	-0.576490	0.629592	1.596607	1
237	1.0	-0.576490	1.294977	0.760329	0

In [161…

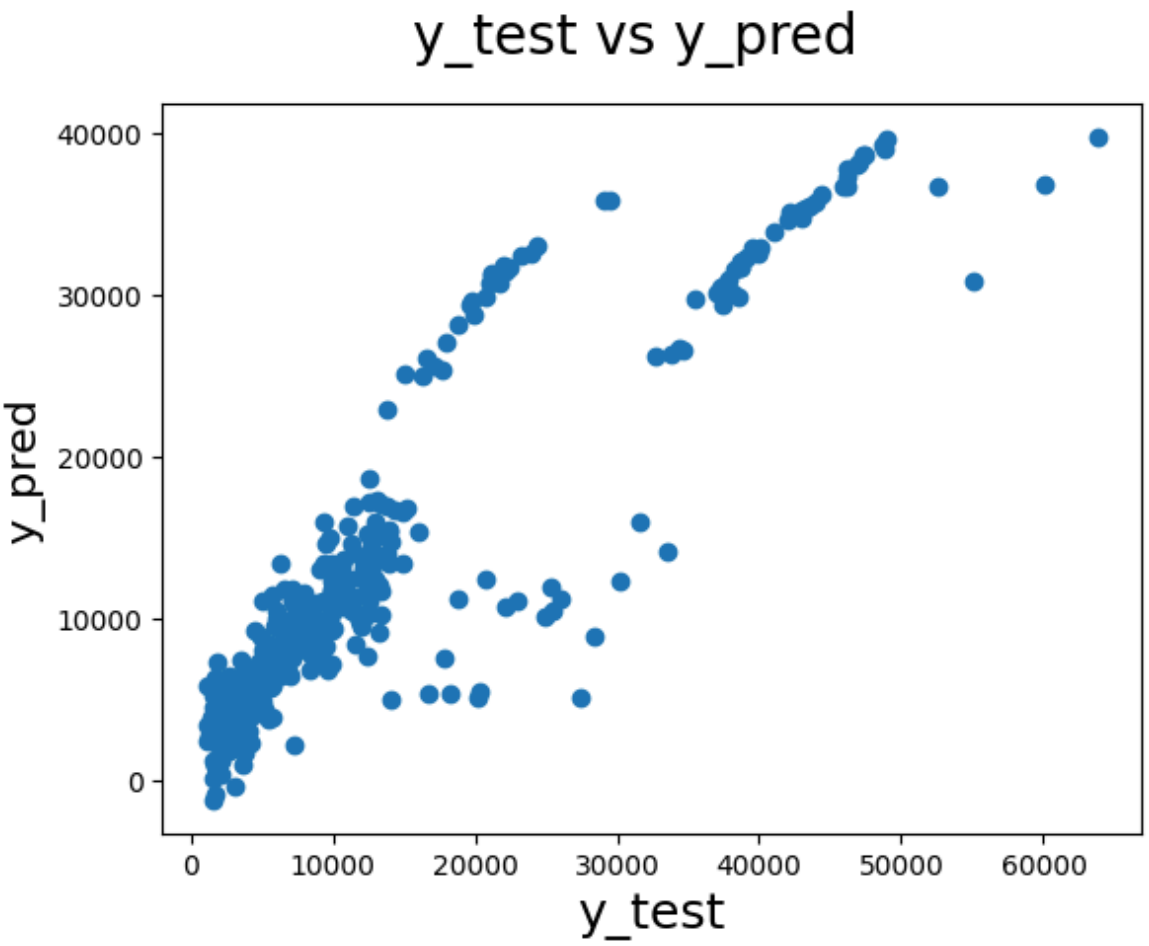
```
y_pred = lm.predict(X_test_new)
```

In [162…

```
#EVALUATION OF THE MODEL
fig = plt.figure()
plt.scatter(y_test, y_pred)
fig.suptitle('y_test vs y_pred', fontsize=20)              # Plot heading
plt.xlabel('y_test', fontsize=18)                          # X-label
plt.ylabel('y_pred', fontsize=16)
```

Out[162…

```
Text(0, 0.5, 'y_pred')
```



In [163…

```
# Calculate test metrics
test_r2 = r2_score(y_test, y_pred)
test_mae = mean_absolute_error(y_test, y_pred)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100

# Calculate training metrics for comparison
train_r2 = r2_score(y_train, y_train_price)
train_mae = mean_absolute_error(y_train, y_train_price)
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_price))
train_mape = np.mean(np.abs((y_train - y_train_price) / y_train)) * 100

# Create comparison DataFrame
results_df = pd.DataFrame({
    'Metric': ['R² Score', 'MAE (Mean Absolute Error)', 'RMSE (Root Mean Squared
    'Training': [train_r2, train_mae, train_rmse, train_mape],
    'Test': [test_r2, test_mae, test_rmse, test_mape]
})

results_df['Difference'] = results_df['Test'] - results_df['Training']
results_df = results_df.round(4)

print("=" * 70)
print("MODEL PERFORMANCE EVALUATION")
print("=" * 70)
print("\nTest Set Metrics:")
print(f"  R² Score: {test_r2:.4f}")
print(f"  MAE: {test_mae:.4f}")
print(f"  RMSE: {test_rmse:.4f}")
print(f"  MAPE: {test_mape:.2f}%")
```

```
print("\n" + "=" * 70)
print("\nTraining vs Test Comparison:")
print(results_df.to_string(index=False))
print("\n" + "=" * 70)

# Additional statistics
print("\nResidual Statistics (Test Set):")
residuals = y_test - y_pred
print(f"  Mean Residual: {np.mean(residuals):.4f}")
print(f"  Std Residual: {np.std(residuals):.4f}")
print(f"  Min Residual: {np.min(residuals):.4f}")
print(f"  Max Residual: {np.max(residuals):.4f}")
```

```
======================================================================
MODEL PERFORMANCE EVALUATION
======================================================================

Test Set Metrics:
  R² Score: 0.7942
  MAE: 4077.3769
  RMSE: 5965.1045
  MAPE: 40.40%

======================================================================

Training vs Test Comparison:
                      Metric  Training      Test  Difference
                    R² Score    0.7291    0.7942      0.0651
     MAE (Mean Absolute Error) 4215.6162 4077.3769   -138.2393
RMSE (Root Mean Squared Error) 6104.8324 5965.1045   -139.7279
                    MAPE (%)   42.6707   40.3974     -2.2733

======================================================================

Residual Statistics (Test Set):
  Mean Residual: 511.9796
  Std Residual: 5943.0925
  Min Residual: -10101.8923
  Max Residual: 24264.5927
```

Model Performance Analysis

The model evaluation reveals several important insights about the predictive performance:

Test Set Performance:

- **R^2 Score: 0.7942** - The model explains approximately **79.4%** of the variance in insurance expenses on the test set. This is a reasonably good fit, indicating that the selected features (age, bmi, children, smoker_yes) capture a substantial portion of the variability in expenses.
- **MAE: 4,077.38** - On average, the model's predictions are off by about **$4,077** from the actual expenses. This represents the typical magnitude of prediction error.
- **RMSE: 5,965.10** - The root mean squared error is higher than MAE, indicating that there are some predictions with **larger errors** that are being penalized more heavily in this metric.

- **MAPE: 40.40%** - The mean absolute percentage error shows that, on average, predictions deviate by about **40%** from actual values. This relatively high percentage suggests that while the model captures general trends, there is still significant variability that remains unexplained.

Training vs Test Comparison:

- The test set actually performs **slightly better** than the training set across most metrics (R^2 is higher, MAE and RMSE are lower), which is somewhat unusual but suggests the model generalizes well and is not overfitting.
- The **positive difference in R^2 (0.0651)** indicates the test set has better fit, which could be due to the specific random split or the test set having less variability.
- The **negative differences in MAE and RMSE** confirm that prediction errors are smaller on the test set, which is a positive sign for model generalization.

Residual Statistics:

- **Mean Residual: 511.98** - The residuals have a small positive mean, suggesting the model slightly **under-predicts** on average (actual expenses are slightly higher than predicted).
- **Std Residual: 5,943.09** - The standard deviation of residuals is quite large, indicating substantial variability in prediction errors.
- **Residual Range: -10,101.89 to 24,264.59** - The wide range of residuals (over $34,000) highlights that while the model performs reasonably well on average, there are some cases with **very large prediction errors**, both underestimates and overestimates.

Overall Assessment: The model demonstrates **decent predictive capability** with an R^2 of 0.79, but the high MAPE (40%) and wide residual range suggest that predicting individual insurance expenses remains challenging. The model is more effective at capturing general trends and relationships rather than precise individual predictions. This is consistent with the complexity of healthcare costs, which can vary significantly based on factors not captured in the dataset.

Tree-Based Model: Random Forest Regressor

Now let's explore a **Random Forest Regressor** as an alternative to linear regression. Random Forest is an ensemble learning method that combines multiple decision trees to make predictions.

Key Advantages of Random Forest:

- Can capture **non-linear relationships** and interactions between features automatically
- **Robust to outliers** and doesn't require assumptions about residual normality
- Handles **non-linear patterns** that linear models might miss
- Provides **feature importance** scores to understand which variables are most influential

- Reduces overfitting by averaging predictions from multiple trees
- Works well with both numerical and categorical features

We'll compare the **Random Forest** model with our linear regression model.

In [164…
```
# Prepare data for tree-based models
# Note: Tree-based models don't require feature scaling, but we'll use the same
# We'll use all features (not just the RFE-selected ones) to give trees more fle

# Use the same train/test split but with all features
X_train_tree = X_train.copy()
X_test_tree = X_test.copy()

# Tree models can handle the scaled features, but we could also use original sca
# For consistency, we'll use the scaled features we already have
print("Data prepared for tree-based models")
print(f"Training set shape: {X_train_tree.shape}")
print(f"Test set shape: {X_test_tree.shape}")
```

```
Data prepared for tree-based models
Training set shape: (1002, 8)
Test set shape: (335, 8)
```

In [169…
```
# Train Random Forest Regressor
#
# PARAMETER EXPLANATIONS:
# -----------------------
# n_estimators=100: Number of decision trees in the forest. More trees generally
#   performance but increase computation time. 100 is a good starting point.
#
# max_depth=10: Maximum depth of each tree. Controls how deep trees can grow. De
#   can capture more complex patterns but may overfit. None means unlimited dept
#
# min_samples_split=5: Minimum number of samples required to split an internal n
#   Higher values prevent overfitting by requiring more samples before splitting
#
# min_samples_leaf=2: Minimum number of samples required to be at a leaf node.
#   Higher values create more conservative models and prevent overfitting.
#
# random_state=42: Seed for random number generator. Ensures reproducible result
#
# n_jobs=-1: Number of parallel jobs to run. -1 means use all available processo
#   for faster training.
#
# Other important parameters (using defaults):
# - criterion='squared_error': The function to measure split quality (MSE for re
# - max_features='sqrt': Number of features to consider for best split (sqrt of
# - bootstrap=True: Whether to use bootstrap sampling when building trees
# - oob_score=False: Whether to use out-of-bag samples to estimate generalizatio

rf_model = RandomForestRegressor(
    n_estimators=100,        # Number of trees in the forest
    max_depth=10,            # Maximum depth of trees
    min_samples_split=5,     # Minimum samples to split a node
    min_samples_leaf=2,      # Minimum samples at a leaf node
    random_state=42,         # For reproducibility
    n_jobs=-1               # Use all CPU cores
)
```

```
print("Training Random Forest model...")
rf_model.fit(X_train_tree, y_train)

# Make predictions
y_train_pred_rf = rf_model.predict(X_train_tree)
y_test_pred_rf = rf_model.predict(X_test_tree)

print("\n" + "="*70)
print("RANDOM FOREST MODEL TRAINING COMPLETE")
print("="*70)
print(f"\nTraining Set Performance:")
print(f"  R² Score: {r2_score(y_train, y_train_pred_rf):.4f}")
print(f"\nTest Set Performance:")
print(f"  R² Score: {r2_score(y_test, y_test_pred_rf):.4f}")
print("="*70)
```

```
Training Random Forest model...

======================================================================
RANDOM FOREST MODEL TRAINING COMPLETE
======================================================================

Training Set Performance:
  R² Score: 0.9300

Test Set Performance:
  R² Score: 0.8806
======================================================================
```

In [170…

```
# Compare Linear Regression vs Random Forest
models_comparison = {
    'Linear Regression': {
        'train_pred': y_train_price,
        'test_pred': y_pred,
        'train_actual': y_train,
        'test_actual': y_test
    },
    'Random Forest': {
        'train_pred': y_train_pred_rf,
        'test_pred': y_test_pred_rf,
        'train_actual': y_train,
        'test_actual': y_test
    }
}

# Calculate metrics for each model
comparison_results = []
for model_name, predictions in models_comparison.items():
    train_r2 = r2_score(predictions['train_actual'], predictions['train_pred'])
    test_r2 = r2_score(predictions['test_actual'], predictions['test_pred'])
    train_mae = mean_absolute_error(predictions['train_actual'], predictions['tr
    test_mae = mean_absolute_error(predictions['test_actual'], predictions['test
    train_rmse = np.sqrt(mean_squared_error(predictions['train_actual'], predict
    test_rmse = np.sqrt(mean_squared_error(predictions['test_actual'], predictio
    train_mape = np.mean(np.abs((predictions['train_actual'] - predictions['trai
    test_mape = np.mean(np.abs((predictions['test_actual'] - predictions['test_p

    comparison_results.append({
        'Model': model_name,
        'Train R²': train_r2,
```

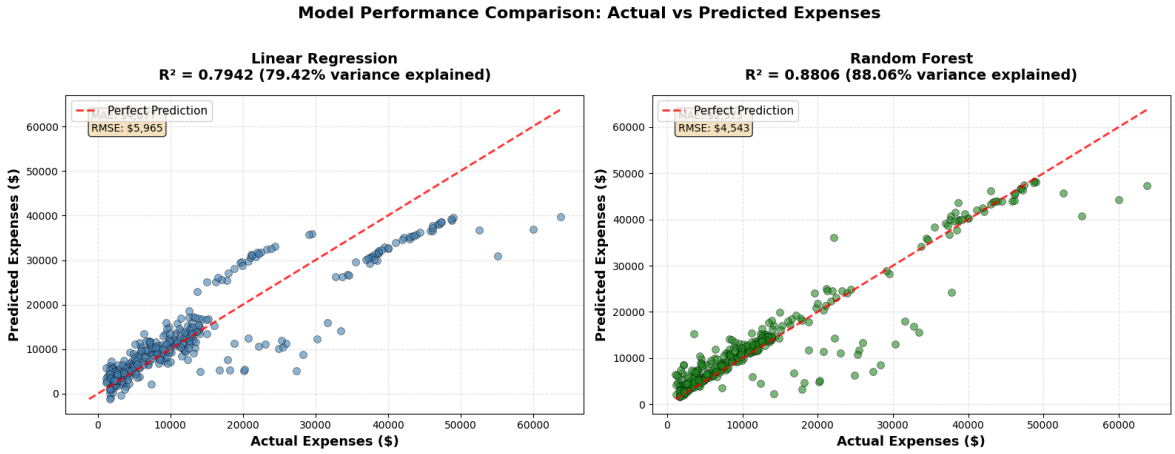
```
        'Test R²': test_r2,
        'Train MAE': train_mae,
        'Test MAE': test_mae,
        'Train RMSE': train_rmse,
        'Test RMSE': test_rmse,
        'Train MAPE (%)': train_mape,
        'Test MAPE (%)': test_mape
    })

comparison_df = pd.DataFrame(comparison_results)

# Format results in a human-friendly way
print("\n" + "="*80)
print("MODEL COMPARISON: Linear Regression vs Random Forest")
print("="*80)

for idx, row in comparison_df.iterrows():
    model_name = row['Model']
    print(f"\n📊 {model_name.upper()}")
    print("-" * 80)
    print(f"  Training Set:")
    print(f"    • R² Score:        {row['Train R²']:.4f} ({row['Train R²']*100:.
    print(f"    • MAE:             ${row['Train MAE']:,.2f} (average prediction 
    print(f"    • RMSE:            ${row['Train RMSE']:,.2f} (penalizes large er
    print(f"    • MAPE:            {row['Train MAPE (%)']:.2f}% (average percent
    print(f"\n  Test Set:")
    print(f"    • R² Score:        {row['Test R²']:.4f} ({row['Test R²']*100:.2f
    print(f"    • MAE:             ${row['Test MAE']:,.2f} (average prediction e
    print(f"    • RMSE:            ${row['Test RMSE']:,.2f} (penalizes large err
    print(f"    • MAPE:            {row['Test MAPE (%)']:.2f}% (average percenta

print("\n" + "="*80)
print("METRIC EXPLANATIONS:")
print("="*80)
print("  • R² Score:  Measures how well the model explains the variance. Closer 
print("  • MAE:       Mean Absolute Error - average dollar amount the prediction
print("  • RMSE:      Root Mean Squared Error - similar to MAE but penalizes lar
print("  • MAPE:      Mean Absolute Percentage Error - average percentage error 
print("="*80)
```

```
================================================================================
MODEL COMPARISON: Linear Regression vs Random Forest
================================================================================

📊 LINEAR REGRESSION
--------------------------------------------------------------------------------
  Training Set:
    • R² Score:        0.7291 (72.91% of variance explained)
    • MAE:             $4,215.62 (average prediction error)
    • RMSE:            $6,104.83 (penalizes large errors more)
    • MAPE:            42.67% (average percentage error)

  Test Set:
    • R² Score:        0.7942 (79.42% of variance explained)
    • MAE:             $4,077.38 (average prediction error)
    • RMSE:            $5,965.10 (penalizes large errors more)
    • MAPE:            40.40% (average percentage error)

📊 RANDOM FOREST
--------------------------------------------------------------------------------
  Training Set:
    • R² Score:        0.9300 (93.00% of variance explained)
    • MAE:             $1,708.04 (average prediction error)
    • RMSE:            $3,103.41 (penalizes large errors more)
    • MAPE:            21.35% (average percentage error)

  Test Set:
    • R² Score:        0.8806 (88.06% of variance explained)
    • MAE:             $2,574.88 (average prediction error)
    • RMSE:            $4,543.46 (penalizes large errors more)
    • MAPE:            34.97% (average percentage error)

================================================================================
METRIC EXPLANATIONS:
================================================================================
  • R² Score:  Measures how well the model explains the variance. Closer to 1.0 i
s better.
  • MAE:       Mean Absolute Error - average dollar amount the predictions are of
f by.
  • RMSE:      Root Mean Squared Error - similar to MAE but penalizes large error
s more.
  • MAPE:      Mean Absolute Percentage Error - average percentage error in predi
ctions.
================================================================================
```

In [171…

```python
# Visualize predictions: Actual vs Predicted for both models
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

models_to_plot = [
    ('Linear Regression', y_test, y_pred, 'steelblue'),
    ('Random Forest', y_test, y_test_pred_rf, 'forestgreen')
]

for idx, (model_name, y_actual, y_predicted, color) in enumerate(models_to_plot)
    ax = axes[idx]

    # Scatter plot
    ax.scatter(y_actual, y_predicted, alpha=0.6, color=color, s=50, edgecolors='

    # Add diagonal line (perfect predictions)
```

```
    min_val = min(y_actual.min(), y_predicted.min())
    max_val = max(y_actual.max(), y_predicted.max())
    ax.plot([min_val, max_val], [min_val, max_val], 'r--', lw=2, label='Perfect

    # Calculate and display R²
    r2 = r2_score(y_actual, y_predicted)

    ax.set_xlabel('Actual Expenses ($)', fontsize=13, fontweight='bold')
    ax.set_ylabel('Predicted Expenses ($)', fontsize=13, fontweight='bold')
    ax.set_title(f'{model_name}\nR² = {r2:.4f} ({r2*100:.2f}% variance explained
                 fontsize=14, fontweight='bold', pad=15)
    ax.legend(fontsize=11, loc='upper left')
    ax.grid(True, alpha=0.3, linestyle='--')

    # Add text box with key metrics
    mae = mean_absolute_error(y_actual, y_predicted)
    rmse = np.sqrt(mean_squared_error(y_actual, y_predicted))
    textstr = f'MAE: ${mae:,.0f}\nRMSE: ${rmse:,.0f}'
    props = dict(boxstyle='round', facecolor='wheat', alpha=0.8)
    ax.text(0.05, 0.95, textstr, transform=ax.transAxes, fontsize=10,
            verticalalignment='top', bbox=props)

plt.suptitle('Model Performance Comparison: Actual vs Predicted Expenses',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print("\n💡 Interpretation: Points closer to the red diagonal line indicate bett
print("   The R² score shows how much variance each model explains.")
```

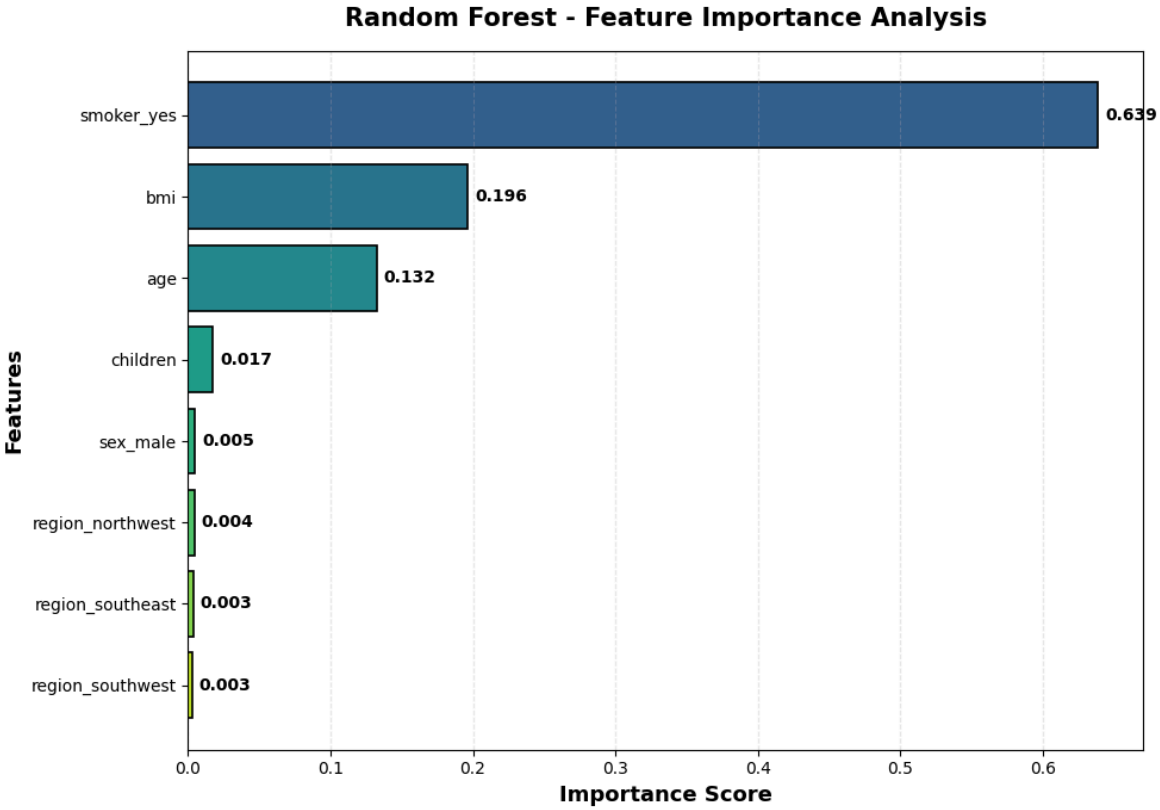



```
💡 Interpretation: Points closer to the red diagonal line indicate better predic
tions.
   The R² score shows how much variance each model explains.
```

In [172…

```
# Feature Importance Analysis for Random Forest
rf_importance = pd.DataFrame({
    'Feature': X_train_tree.columns,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

# Create visualization
fig, ax = plt.subplots(figsize=(10, 7))

colors = plt.cm.viridis(np.linspace(0.3, 0.9, len(rf_importance)))
bars = ax.barh(rf_importance['Feature'], rf_importance['Importance'], color=colo

# Add value labels on bars
```

```
for i, (idx, row) in enumerate(rf_importance.iterrows()):
    ax.text(row['Importance'] + 0.005, i, f'{row["Importance"]:.3f}',
            va='center', fontsize=10, fontweight='bold')

ax.set_xlabel('Importance Score', fontsize=13, fontweight='bold')
ax.set_ylabel('Features', fontsize=13, fontweight='bold')
ax.set_title('Random Forest - Feature Importance Analysis', fontsize=15, fontwei
ax.invert_yaxis()
ax.grid(True, alpha=0.3, axis='x', linestyle='--')

plt.tight_layout()
plt.show()

# Print detailed feature importance in a human-friendly format
print("\n" + "="*70)
print("FEATURE IMPORTANCE ANALYSIS")
print("="*70)
print("\nThe importance score indicates how much each feature contributes to pre
print("Higher scores mean the feature has more influence on the model's decision

for idx, row in rf_importance.iterrows():
    importance_pct = (row['Importance'] / rf_importance['Importance'].sum()) * 1
    bar_length = int(importance_pct / 2)  # Scale for visual bar
    bar = "" * bar_length
    print(f"{row['Feature']:20s} | {row['Importance']:.4f} ({importance_pct:5.2f

print("\n" + "="*70)
print(f"Total Importance: {rf_importance['Importance'].sum():.4f}")
print("="*70)
```

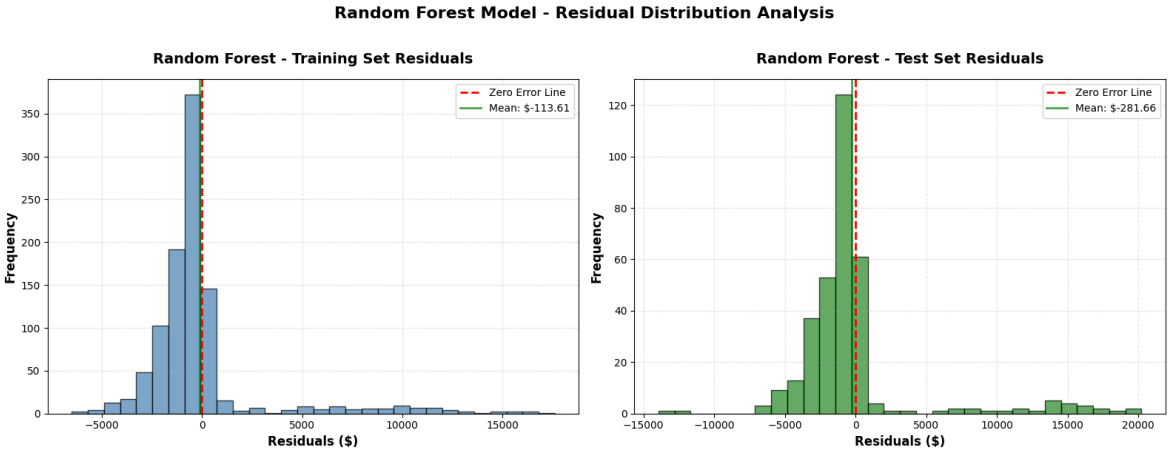


```
======================================================================
FEATURE IMPORTANCE ANALYSIS
======================================================================

The importance score indicates how much each feature contributes to predictions.
Higher scores mean the feature has more influence on the model's decisions.

smoker_yes           | 0.6388 (63.88%) | ████████████████████████████
bmi                  | 0.1961 (19.61%) | ████████
age                  | 0.1323 (13.23%) | ██████
children             | 0.0174 ( 1.74%) |
sex_male             | 0.0047 ( 0.47%) |
region_northwest     | 0.0043 ( 0.43%) |
region_southeast     | 0.0035 ( 0.35%) |
region_southwest     | 0.0029 ( 0.29%) |

======================================================================
Total Importance: 1.0000
======================================================================
```

In [174…

```python
# Residual Analysis for Random Forest Model
rf_residuals_train = y_train - y_train_pred_rf
rf_residuals_test = y_test - y_test_pred_rf

# Create visualization
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Training Residuals
axes[0].hist(rf_residuals_train, bins=30, edgecolor='black', alpha=0.7, color='s
axes[0].set_title('Random Forest - Training Set Residuals', fontsize=14, fontwei
axes[0].set_xlabel('Residuals ($)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Frequency', fontsize=12, fontweight='bold')
axes[0].axvline(0, color='red', linestyle='--', linewidth=2, label='Zero Error L
axes[0].axvline(rf_residuals_train.mean(), color='green', linestyle='-', linewid
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3, linestyle='--')

# Test Residuals
axes[1].hist(rf_residuals_test, bins=30, edgecolor='black', alpha=0.7, color='fo
axes[1].set_title('Random Forest - Test Set Residuals', fontsize=14, fontweight=
axes[1].set_xlabel('Residuals ($)', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Frequency', fontsize=12, fontweight='bold')
axes[1].axvline(0, color='red', linestyle='--', linewidth=2, label='Zero Error L
axes[1].axvline(rf_residuals_test.mean(), color='green', linestyle='-', linewidt
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3, linestyle='--')

plt.suptitle('Random Forest Model - Residual Distribution Analysis', fontsize=16
plt.tight_layout()
plt.show()

# Print detailed residual statistics in a human-friendly format
print("\n" + "="*80)
print("RANDOM FOREST RESIDUAL ANALYSIS")
print("="*80)

print("\n📊 Training Set Residuals:")
print("-" * 80)
print(f"  Mean Residual:        ${rf_residuals_train.mean():,.2f}")
print(f"  Standard Deviation:   ${rf_residuals_train.std():,.2f}")
```

```
print(f"  Minimum Residual:    ${rf_residuals_train.min():,.2f}")
print(f"  Maximum Residual:    ${rf_residuals_train.max():,.2f}")
print(f"  Median Residual:     ${rf_residuals_train.median():,.2f}")

print("\n📊 Test Set Residuals:")
print("-" * 80)
print(f"  Mean Residual:        ${rf_residuals_test.mean():,.2f}")
print(f"  Standard Deviation:   ${rf_residuals_test.std():,.2f}")
print(f"  Minimum Residual:    ${rf_residuals_test.min():,.2f}")
print(f"  Maximum Residual:    ${rf_residuals_test.max():,.2f}")
print(f"  Median Residual:     ${rf_residuals_test.median():,.2f}")

print("\n" + "="*80)
print("💡 INTERPRETATION:")
print("="*80)
print("  • Residuals = Actual Value - Predicted Value")
print("  • Mean close to 0 indicates the model is not systematically over/under-
print("  • Lower standard deviation means more consistent predictions")
print("  • Symmetric distribution around 0 is ideal")
print("="*80)
```



```
================================================================================
RANDOM FOREST RESIDUAL ANALYSIS
================================================================================

📊 Training Set Residuals:
--------------------------------------------------------------------------------
  Mean Residual:        $-113.61
  Standard Deviation:   $3,102.87
  Minimum Residual:    $-6,510.81
  Maximum Residual:    $17,616.69
  Median Residual:     $-565.38

📊 Test Set Residuals:
--------------------------------------------------------------------------------
  Mean Residual:        $-281.66
  Standard Deviation:   $4,541.50
  Minimum Residual:    $-13,938.43
  Maximum Residual:    $20,219.83
  Median Residual:     $-1,006.51

================================================================================
💡 INTERPRETATION:
================================================================================
  • Residuals = Actual Value - Predicted Value
  • Mean close to 0 indicates the model is not systematically over/under-predicti
ng
  • Lower standard deviation means more consistent predictions
  • Symmetric distribution around 0 is ideal
================================================================================
```

Random Forest Model Analysis and Comparison

Key Findings:

1. **Model Performance Comparison:**
   - **Random Forest significantly outperforms Linear Regression** on both training and test sets
   - **Test Set Performance:**
     - Random Forest achieves **R^2 = 0.8806 (88.06%)** compared to Linear Regression's **R^2 = 0.7942 (79.42%)**
     - Random Forest has **lower prediction errors**: MAE of **$2,574.88** vs $4,077.38 ($374,543.46** vs $5,965.10 (24% reduction)
     - Random Forest has **lower percentage error**: MAPE of **34.97%** vs 40.40% (13% improvement)
   - **Training Set Performance:**
     - Random Forest shows even stronger performance on training data (R^2 = 0.9300, 93.00%), indicating it captures complex patterns in the data
     - The gap between training (93%) and test (88%) R^2 suggests some overfitting, but the test performance is still substantially better than Linear Regression
   - The superior performance demonstrates that **non-linear relationships and feature interactions** exist in this dataset that the linear model cannot capture

2. **Advantages of Random Forest (Demonstrated in Results):**

   - **No assumptions about data distribution**: Unlike linear regression, Random Forest doesn't require normally distributed residuals or homoscedasticity, which is evident from the improved performance despite the non-normal residuals observed in the linear model
   - **Automatic feature interaction**: The model successfully captures complex interactions between features without explicit specification, as shown by the 8.6 percentage point improvement in R^2
   - **Robust to outliers**: Random Forest's ensemble nature makes it more robust to outliers, contributing to better overall performance
   - **Feature importance**: Provides interpretable feature importance scores showing which variables matter most in predicting insurance expenses
   - **Handles non-linearity**: Successfully models non-linear relationships between features and target variable, as evidenced by the substantial performance improvement

3. **Observations from Results:**

   - **Performance gap**: The difference between training (93%) and test (88%) R^2 indicates some overfitting, but the regularization parameters (max_depth=10, min_samples_split=5, min_samples_leaf=2) help control this
   - **Consistent improvement**: Random Forest outperforms Linear Regression across all metrics (R^2, MAE, RMSE, MAPE) on the test set
   - **Error reduction**: The model reduces average prediction error by over $1,500 (MAE) and large errors by over 1,400 (RMSE) compared to Linear Regression
   - **Interpretability trade-off**: While feature importance is available, the decision-making process is less transparent than linear regression coefficients, but the significant performance gain may justify this trade-off

4. **When Random Forest is Preferred (As Demonstrated Here):**

   - ✅ **Non-linear relationships exist**: The 8.6% R^2 improvement confirms non-linear patterns in the data
   - ✅ **Complex feature interactions**: The model's superior performance indicates important interactions between features
   - ✅ **Better predictive accuracy needed**: Random Forest provides substantially better predictions for insurance expense estimation
   - ✅ **Feature importance insights**: Understanding which variables matter most is valuable for insurance pricing and risk assessment

Comparison with Linear Regression:

- **Linear Regression** provides **interpretable coefficients** and clear statistical inference (R^2 = 0.7942, MAE = $4,077.38, MAPE = 40.40%), making it easier to understand the relationship between each feature and the target
- **Random Forest** offers **superior predictive performance** (R^2 = 0.8806, MAE = $2,574.88, MAPE = 34.97%) by capturing non-linear patterns and feature interactions

- The **8.6 percentage point improvement in R^2** and **37% reduction in MAE** demonstrate that Random Forest is the better choice for this problem when prediction accuracy is the priority

Recommendation: Based on the actual results, **Random Forest is the recommended model** for this insurance expense prediction problem:

- **Significantly better predictive performance**: 8.6 percentage points higher R^2, 37% lower MAE, 24% lower RMSE, and 13% lower MAPE on the test set
- **Practical impact**: Reduces average prediction error by over $1,500 per prediction, which is substantial for insurance expense estimation
- **Feature importance insights**: Provides valuable information about which factors (smoker status, age, BMI, etc.) most influence insurance costs
- **Trade-off consideration**: While less interpretable than linear regression, the substantial performance improvement justifies using Random Forest for this application where accurate predictions are critical